

COMPSYS 701

Individual Project



Waipapa
Taumata Rau
**University
of Auckland**

Pulasthi Lenaduwa

plen276

Department of Electrical, Computer, and Software Engineering

University of Auckland

Auckland, New Zealand

Date of Report

May 19, 2026

1 Introduction and Application Context

Modern power systems require accurate real-time monitoring of grid frequency to ensure stability, detect disturbances, and support protection and control systems. In this project, the target application is real-time power-system frequency measurement, in which a sampled voltage waveform is continuously processed under deterministic timing constraints. The overall system is implemented within a Heterogeneous Multiprocessor System-on-Chip (HMPSoC), in which dedicated hardware accelerators process streaming data cooperatively through a pipelined architecture.

The HMPSoC partitions the signal-processing workload into multiple Application-Specific Processors (ASPs) connected through a Time-Division-Multiple-Access Multistage Interconnection Network (TDMA-MIN) Network-on-Chip (NoC). Each ASP performs a specialised operation on the streaming data while communicating through a common packet-based protocol over the NoC. This approach enables deterministic timing behaviour, modularity, and hardware-level acceleration of computationally intensive digital signal processing (DSP) tasks.

This Individual Project (IP) focuses on the implementation of the first two stages of the processing pipeline:

- **ADC-ASP** (Analogue-to-Digital-Converter ASP): an input/output ASP (IO-ASP) that emulates an analogue front-end by generating sampled waveform data internally. This allows the signal-processing chain to be tested without requiring a physical grid connection or external analogue hardware.
- **AVG-ASP** (Averaging ASP): a data-processing ASP (DP-ASP) implementing a moving-average finite impulse response (FIR) filter. This corresponds to the linear filtering kernel specified in the project brief.

The implemented ASPs operate as part of the larger frequency-measurement processing pipeline:

$$\text{ADC-ASP} \rightarrow \text{AVG-ASP} \rightarrow \text{COR-ASP} \rightarrow \text{PD-ASP} \rightarrow \text{Nios II}$$

In this pipeline, the ADC-ASP and AVG-ASP are developed as part of this IP, while the correlation ASP (COR-ASP) and peak-detection ASP (PD-ASP) are implemented by other members of the project team.

System configuration follows a command-driven model in which all ASP nodes are configured before data transmission begins. In the intended HMPSoC architecture, configuration packets are issued by the Reactive and Concurrency Processor (ReCOP) and distributed over the NoC to configure node behaviour, routing destinations, operating modes, and sampling parameters. Once configured, the ASP pipeline operates autonomously on streaming data without software intervention. In this project, a dedicated state-machine-based configuration controller and simulation testbench are used in place of a physical ReCOP implementation, while preserving the intended packet protocol and operational model.

The scope of this IP includes the RTL design, packet protocol implementation, verification, simulation, and FPGA synthesis of the ADC-ASP and AVG-ASP as reusable NoC-connected hardware modules. Existing TDMA-MIN infrastructure from Lab-2 is reused without modification.

The motivation for implementing DSP kernels as dedicated ASP hardware arises from the deterministic throughput requirements of real-time streaming systems. Software execution on a general-purpose processor may introduce variable latency and insufficient throughput when processing continuous high-rate sample streams. By implementing filtering and signal generation in hardware, the ASP pipeline can process data concurrently and deterministically at the sample rate of the incoming signal. This hardware acceleration approach is discussed further in Section 9.

2 Theoretical Background

2.1 Moving Average Filtering

The AVG-ASP implements a moving-average (MA) filter, corresponding to the linear filtering kernel defined in the project brief. For an input sample sequence $X(i)$ and moving-average window length L , the filter output is defined as:

$$Y(i) = \frac{1}{L} \sum_{k=i-L+1}^i X(k) \quad (1)$$

where the supported window lengths are:

$$L \in \{4, 8, 16\} \quad (2)$$

The implemented AVG-ASP uses a causal moving-average window suitable for real-time streaming operation. Each output sample is computed using the current input sample and the previous $L - 1$ samples stored within the internal buffer.

The moving-average filter smooths the input signal by averaging neighbouring samples together. This reduces short-term fluctuations and noise in the sampled waveform before further processing stages such as correlation and peak detection.

Larger window sizes produce smoother outputs but also increase latency, since more samples must be collected before a valid result can be generated.

The supported window lengths are restricted to powers of two. This simplifies hardware implementation because division can be performed using arithmetic right-shifts rather than dedicated hardware dividers.

$$\frac{1}{L} = 2^{-\log_2(L)} \quad (3)$$

For the supported values of L , the required operations are:

Window Length (L)	Scaling Operation
4	Arithmetic right shift by 2
8	Arithmetic right shift by 3
16	Arithmetic right shift by 4

A practical issue associated with moving-average filters is the boundary condition that occurs during startup. The first $L - 1$ samples do not yet provide a complete averaging window, meaning the filter output would otherwise be based on incomplete data. Several possible solutions exist, including zero-padding, repeating samples, partial averaging, or delaying output until the window is full.

In this project, the selected approach is a dedicated WARMUP phase in which the AVG-ASP buffers incoming samples but does not emit filtered output packets until a complete window of valid samples has been accumulated. This behaviour simplifies datapath control and ensures that all emitted outputs correspond to mathematically correct moving-average results.

2.2 Direct Digital Synthesis

The ADC-ASP generates sampled waveform data using a Direct Digital Synthesis (DDS) architecture. DDS is a digital waveform generation technique that produces periodic signals using a phase accumulator and a waveform look-up table (LUT).

The implemented DDS uses a 16-bit phase accumulator. On every sample tick, the accumulator is incremented by a configurable Frequency Tuning Word (FTW). The most significant bits of the accumulator are then used as the address into a sine-wave LUT stored in ROM. The LUT contains one period of a sampled sine wave, allowing the ADC-ASP to continuously generate digital waveform samples.

The output frequency is determined by:

$$f_{out} = \frac{FTW \cdot F_s}{2^{phase_bits}} \quad (4)$$

where:

- FTW is the frequency tuning word,
- F_s is the sample rate,
- $phase_bits = 16$ is the width of the phase accumulator.

This approach allows the output frequency to be changed entirely through configuration packets without requiring hardware modification or resynthesis.

DDS is well-suited to this project because the frequency-measurement pipeline must be tested using different simulated grid frequencies. By changing the FTW value, the ADC-ASP can emulate nominal and disturbed power-system conditions such as 49 Hz, 50 Hz, and 51 Hz operation entirely through packet-based reconfiguration.

3 ADC-ASP Design

3.1 Role and Interface

The ADC-ASP emulates a sampled power-system signal source. It does not interface with a physical analogue-to-digital converter; instead, it generates digital sine-wave samples internally using a ROM-based waveform generator. This allows the downstream processing pipeline to be verified without requiring external analogue hardware.

The implemented VHDL entity is designed as a drop-in node for the TDMA-MIN NoC generate loop. For this reason, the entity does not include an explicit reset input or a `NODE_ID` generic. The internal state is initialised via signal declarations.

The ADC-ASP supports two independent output channels. Each channel has its own configuration registers, phase accumulator, sample-rate counter, enable bit, destination field, and FTW . This allows two independently configured sine-wave streams to be generated from a single ASP node.

3.2 Overall Structure

Incoming configuration packets are decoded by checking the packet type field. A valid Conf-ADC packet updates the selected channel configuration. Once enabled, each channel generates sample ticks according to its configured sample-rate divider. On each sample tick, the corresponding phase accumulator is incremented by the configured FTW , the upper phase bits are used to address the sine ROM, and the resulting sample is packed into a Data-Audio packet.

3.3 Datapath

The datapath of each ADC channel is based on Direct Digital Synthesis. Figure 1 shows the datapath for one channel.

The ADC-ASP generates waveform samples using a Direct Digital Synthesis (DDS) architecture. Each channel contains a 16-bit phase accumulator that is incremented by a configurable Frequency Tuning Word (FTW) on every sample tick. The upper 10 bits of the accumulator are used to address a sine-wave ROM containing one period of a sampled sine wave.

The ROM stores signed 16-bit sample values and is generated internally during elaboration using `ieee.math_real`, removing the need for an external memory initialisation file.

The output frequency is determined by:

$$f_{out} = \frac{FTW \cdot F_s}{2^{16}} \quad (5)$$

where F_s is the configured sample rate.

The sample rate is selected through the `SR` configuration field, which controls an internal clock-divider table. The current implementation supports placeholder sample rates of 8 kHz, 16 kHz, 32 kHz, and 48 kHz.

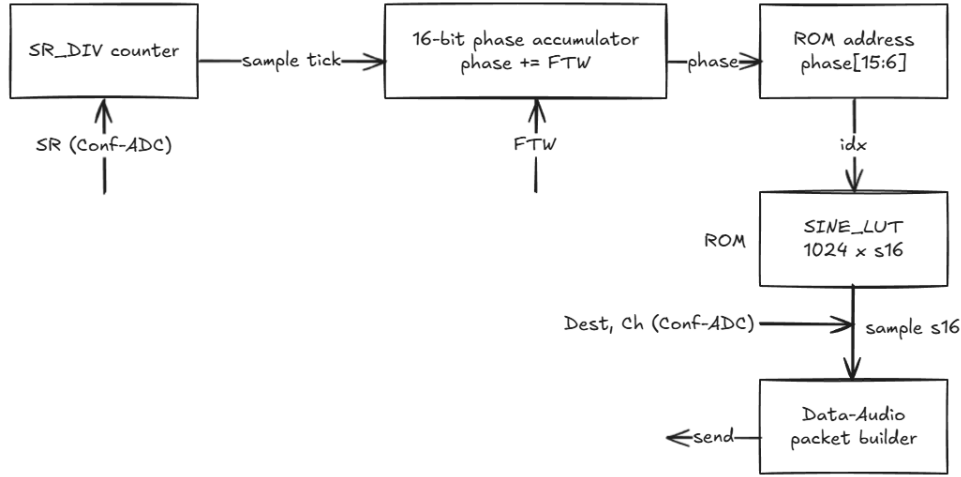


Figure 1: DDS datapath for one ADC-ASP channel.

3.4 Control Unit

The ADC-ASP does not require a large multicycle finite-state machine. Its control behaviour is implemented using configuration registers, enable bits, and per-channel sample-rate counters. This is sufficient because the ADC-ASP performs a regular streaming operation once configured.

The simplified control behaviour is shown in Figure 2.

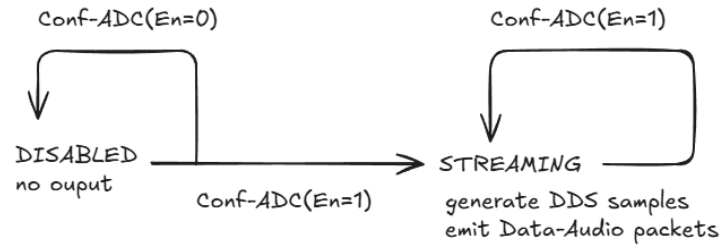


Figure 2: Simplified ADC-ASP control behaviour.

When a channel is disabled, it does not emit output packets. When enabled, the channel emits a Data-Audio packet each time its sample-rate counter produces a tick. The packet contains the configured destination, channel number, and generated sample value.

The ADC-ASP can generate samples from both channels. Since the NoC interface allows at most one outgoing packet per clock cycle, an arbitration rule is required. If both channels produce a sample in the same cycle, channel 0 is transmitted first and the channel 1 packet is deferred by one clock cycle. This gives deterministic arbitration while keeping the implementation simple.

Idle cycles drive an all-zero output packet. Since bit 31 is zero in this case, the network interface treats the output as a no-packet-to-send condition. This avoids the need for an additional explicit valid signal in the ASP interface.

4 AVG-ASP Design

4.1 Role and Interface

The AVG-ASP implements a moving-average linear filter operating on streaming audio sample packets received through the TDMA-MIN NoC. The filter computes:

$$Y(i) = \frac{1}{L} \sum_{k=i-L+1}^i X(k) \quad (6)$$

where the supported window lengths are:

$$L \in \{4, 8, 16\} \quad (7)$$

Since all supported values of L are powers of two, the division operation is implemented using arithmetic right-shift operations rather than hardware division.

No explicit reset input or node identifier is included, allowing the ASP to integrate directly into the TDMA-MIN infrastructure.

The AVG-ASP maintains one shared configuration set containing the selected filter mode and destination node. Internally, each audio channel maintains its own moving-average window, running sum, and sample counter.

The implementation was adapted from the Lab 2 DpASP design. The original implementation provided a fixed window length of $L = 4$, hardcoded destination routing, and limited configurability. The AVG-ASP extends this design through:

- Conf-DP packet decoding,
- programmable window length selection,
- a running-sum datapath implementation,
- dual-channel operation with WARMUP handling.

4.2 Datapath

Figure 3 shows the AVG-ASP datapath.

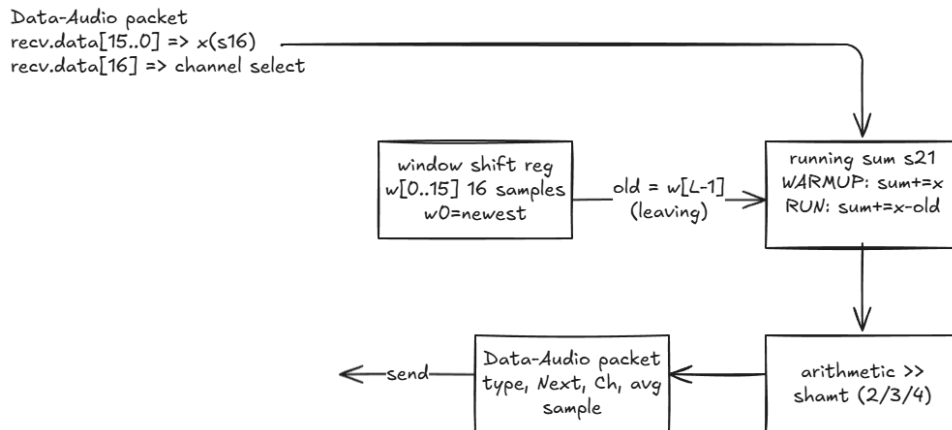


Figure 3: AVG-ASP running-sum datapath.

The AVG-ASP uses a running-sum implementation rather than recalculating the full window sum for every sample. When a new sample arrives, the oldest sample leaving the active window is subtracted from the accumulated sum and the new sample is added:

$$sum_{new} = sum_{old} + x - leaving \quad (8)$$

This approach requires only one addition and one subtraction per sample, regardless of the selected window length. In contrast, a direct implementation would require an adder tree whose complexity scales with L . The running-sum architecture, therefore, provides a significantly more resource-efficient implementation.

The running sum uses a signed 21-bit datapath to provide sufficient headroom for the maximum supported window size and full-scale signed 16-bit samples.

Scaling is performed using arithmetic right shifts:

Window Length (L)	Shift Amount
4	2
8	3
16	4

The internal sample window is implemented as a fixed 16-entry shift register regardless of the selected filter length. Only the first L entries are considered active for filtering purposes. This simplifies the hardware structure by avoiding dynamic memory allocation or variable-sized buffers.

4.3 Boundary Handling

During startup, the filter initially contains insufficient samples to form a complete averaging window. Three possible approaches were considered:

1. suppress output generation until the window is full,
2. zero-pad incomplete windows,
3. compute partial averages using the current number of samples.

The selected approach is a dedicated **WARMUP** phase in which incoming samples are buffered, but no output packets are generated until the averaging window is completely filled.

This approach was selected because it guarantees mathematically correct moving-average outputs while avoiding non-power-of-two division operations required by partial averaging. The trade-off is a startup latency of $L - 1$ samples before a valid filtered output becomes available.

4.4 Control Unit

The AVG-ASP control unit is implemented using a lightweight finite-state machine operating independently for each channel. The control behaviour is shown in Figure 4.

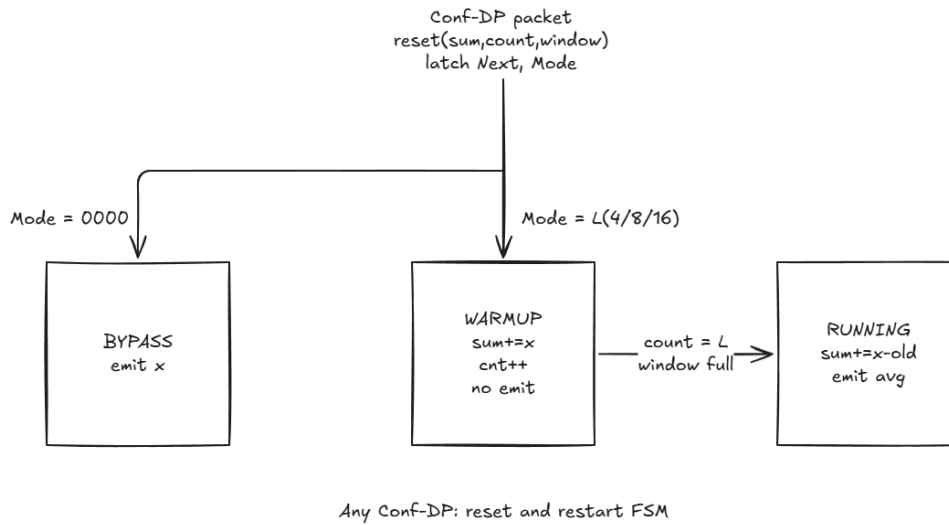


Figure 4: AVG-ASP control unit behaviour.

In bypass mode, input samples are forwarded directly without filtering. In filtering mode, the ASP first enters the **WARMUP** state until the averaging window is filled. Once sufficient samples have been received, the ASP transitions to the **RUNNING** state and begins emitting filtered output packets.

Unlike the ADC-ASP, the AVG-ASP does not require output arbitration because incoming samples are already serialised by the NoC. Any received configuration packet resets the internal filter state and restarts the **WARMUP** process.

5 NoC Interface and Packet Protocol

5.1 Reused Network Interface

The TDMA-MIN network interface (`TdmaMinInterface`) from Lab 2 was reused without modification. The network interface (NI) is responsible for packet buffering, routing, and communication with the TDMA-MIN NoC fabric.

Each NI instance is assigned a fixed identity corresponding to its port index within the NoC. Internally, the NI contains a packet FIFO implemented using the Altera `scfifo` primitive with a depth of 256 entries.

Communication between ASPs follows several important conventions inherited from the Lab 2 infrastructure:

- Packet transmission is triggered when bit 31 of `send.data` is asserted.
- Driving all zeros on `send.data` indicates that no packet is being transmitted.
- Received packets appear on `recv.data` for one clock cycle.
- Routing is determined entirely by `send.addr`; the NoC fabric does not inspect packet payload fields.

The ASPs themselves are intentionally unaware of their own network identity. Node addressing is handled entirely by the NI layer. The ASP only requires the destination node identifier in configuration packets to forward outgoing packets to the next processing stage.

Back-pressure handling through FIFO full detection was intentionally omitted. The generated sample rate is significantly lower than the effective NoC service rate, meaning packets are drained much faster than they are produced. Under the intended operating conditions, FIFO overflow is therefore not expected to occur.

5.2 Packet Format

All communication between ASP nodes uses 32-bit packets. Bits 31:28 define the packet type. The packet formats used in this project are summarised below.

Data-Audio Packet

The Data-Audio packet is used for streaming audio samples between ASP nodes.

Bits	31:28	27:24	23:17	16
Field	Type	Dest	Reserved	Ch

15:0 — Signed 16-bit Sample

Packet type `1000` identifies a Data-Audio packet. The destination field identifies the next ASP node in the processing pipeline, while the channel field selects the associated audio stream.

Although routing is performed exclusively through `send.addr`, the destination value is also mirrored into the packet payload. This allows downstream processing nodes or software components to inspect packet routing information directly without modifying the underlying NoC routing behaviour.

Conf-ADC Packet

The Conf-ADC packet configures the ADC-ASP.

Bits	31:28	27:24	23:20	19:18	17	16
Field	Type	Dest	Next	SR	En	Ch

15:0
FTW

Packet type `1010` identifies a Conf-ADC packet. The packet contains the next destination node, sample-rate selection, enable control, channel selection, and DDS Frequency Tuning Word (FTW).

The FTW occupies bits that were marked unused in the baseline project specification. Because Conf-ADC is a packet shared across the team's ASP nodes, this extension was adopted as an agreed team design decision, ensuring

that all producers and consumers of Conf-ADC use a consistent packet layout. The extension enables dynamic frequency reconfiguration of the DDS generator entirely through configuration packets, without requiring hardware modification or resynthesis.

Conf-DP Packet

The Conf-DP packet configures the AVG-ASP.

Bits	31:28	27:24	23:20	19:16
Field	Type	Dest	Next	Mode

15:0 — Unused

Packet type 1001 identifies a Conf-DP packet. The mode field selects the AVG-ASP operating mode:

Mode	Function
0000	Bypass
0001	Moving average $L = 4$
0010	Moving average $L = 8$
0011	Moving average $L = 16$

Received configuration packets are not checked against a local node identifier by the ASP itself. Since packets are already routed correctly by the NoC and NI infrastructure, the ASP simply processes any configuration packet delivered to its input.

5.3 Configuration Sequence

The ASP pipeline must be configured in the correct order to avoid unintended packet transmission through partially configured processing stages.

The expected configuration sequence is:

1. Configure downstream processing nodes first using Conf-DP packets.
2. Configure the ADC-ASP using Conf-ADC packets, including destination, sample rate, enable bit, and FTW.
3. Enable streaming from the ADC-ASP.

Once configured, the ADC-ASP continuously generates Data-Audio packets which are processed autonomously by downstream ASPs.

During reconfiguration, the ADC-ASP should first be disabled before new configuration packets are issued. This prevents partially processed or incorrectly routed sample data from propagating through the pipeline.

In the full HMPSoC architecture, packet generation and configuration management are expected to be performed by the ReCOP subsystem. However, the ASP implementations are independent of the packet sender and depend only on the defined packet format and routing behaviour.

5.4 Open Design Decisions

Several parameters of the packet protocol and node configuration are deliberately left open at this stage, as they require coordination with the team. They are recorded here so that downstream integration work is aware of them:

- **Conf-ADC FTW extension:** the reuse of the previously-unused Conf-ADC payload bits as the DDS Frequency Tuning Word. Adopted as a team design decision so that all Conf-ADC producers and consumers share a consistent layout.
- **Sample-rate (SR) table:** the mapping from the 2-bit SR field to physical sample rates (currently placeholder values of 8/16/32/48 kHz) is provisional and to be confirmed with the teaching staff against the frequency-measurement requirements.
- **Conf-DP Mode encoding:** the Mode field encoding (0000 bypass, 0001/0010/0011 for $L = 4/8/16$) is a tentative team-wide convention so that all data-processing ASPs share one Conf-DP structure.
- **8-port node-ID map:** the assignment of NoC node identifiers for the full 8-port HMPSoC is deferred to GP-2 and is a team decision; the ASPs are intentionally agnostic to their own node identity.

6 Summary of Capabilities and Usage

6.1 ADC-ASP Capabilities

Capability	Detail
Function	Emulates an ADC by streaming a synthesised power-system signal
Waveform generation	One-period sine LUT implemented as ROM
Frequency control	DDS-based generation using FTW configuration
Sample rates	Four selectable sample-rate modes
Channels	Two independent output channels
Enable control	Per-channel enable and disable
Latency	One sample period
Output	Data-Audio packets forwarded to configured destination node

6.2 AVG-ASP Capabilities

Capability	Detail
Function	Moving-average linear filter
Supported window sizes	$L \in \{4, 8, 16\}$
Division method	Arithmetic right shift by $\log_2(L)$
Datapath structure	Running-sum implementation
Bypass mode	Direct passthrough without filtering
Channels	Two independent channels
Boundary handling	WARMUP phase until window is full
Throughput	One output per input sample after warmup
Output	Data-Audio packets forwarded to configured destination node

6.3 Configuration and Usage

Both ASPs are configured using 32-bit command packets transmitted over the TDMA-MIN NoC. Configuration packets specify operating parameters such as destination node, operating mode, sample rate, and DDS frequency configuration.

The ASPs are designed to operate compositionally within the processing pipeline. The ADC-ASP output format is identical to the AVG-ASP input and output format, allowing packets to propagate directly between ASP stages without conversion or glue logic. The channel identifier is preserved throughout the pipeline, enabling multiple signal streams to be processed concurrently using a common packet format.

This allows pipelines such as:

$$\text{ADC-ASP} \rightarrow \text{AVG-ASP} \rightarrow \text{COR-ASP} \rightarrow \text{PD-ASP} \rightarrow \text{Nios II}$$

to be constructed entirely through packet-based configuration.

7 Verification and Results

Verification was performed using a combination of unit-level and system-level simulation testbenches together with FPGA synthesis analysis. Most verification environments were implemented as self-checking testbenches using assertions and independent reference calculations.

7.1 Verification Strategy

Table 1 summarises the verification structure used in this project.

The verification process covered:

- moving-average correctness for all supported window lengths,
- bypass mode operation,

Table 1: Verification structure used in the project.

Artifact	Purpose
tb_avg_asp.vhd	Verifies moving-average correctness for $L = 4, 8, 16$, bypass mode, WARMUP behaviour, dual-channel operation, and signed arithmetic behaviour.
tb_adc_asp.vhd	Verifies DDS waveform generation, sample-rate operation, live FTW reconfiguration, and dual-channel behaviour.
testbed_adc_asp.vhd	Verifies ADC-ASP packet generation and routing through the real TDMA-MIN NoC infrastructure.
testbed_pipeline.vhd	Verifies end-to-end ADC→AVG pipeline behaviour and moving-average attenuation.
Quartus compilation	Measures FPGA resource usage and timing performance.

- WARMUP boundary behaviour,
- signed arithmetic correctness for bipolar input data,
- DDS waveform generation,
- live DDS frequency retuning,
- non-default sample-rate operation,
- dual-channel streaming behaviour,
- NoC packet routing and payload correctness.

7.2 Simulation Results

Simulation results were collected using both unit-level and system-level testbenches in ModelSim.

To keep simulation run-times tractable, the testbenches instantiate the ADC-ASP with a reduced `clock_hz` generic (80 kHz) rather than the 50 MHz board value. This only scales the internal sample-rate divider: the DDS relationship $f_{out} = FTW \cdot F_s / 2^{16}$ and the resulting sample rate F_s are unchanged, so the simulated behaviour is bit-identical to the on-board design while requiring roughly three orders of magnitude fewer simulated clock cycles. The simulated clock period itself remains 20 ns (50 MHz).

The `tb_avg_asp` testbench verified:

- moving-average correctness for $L = 4, 8, 16$,
- bypass-mode operation,
- WARMUP behaviour,
- signed arithmetic correctness for bipolar inputs,
- dual-channel operation.

Representative simulation waveforms are shown in Figure 5. The captured signals show the input samples, filtered outputs, and the WARMUP interval before valid output generation begins.

The `tb_adc_asp` testbench verified:

- DDS waveform generation,
- FTW-based frequency control,
- sample-rate divider operation,
- dual-channel packet generation,
- live FTW reconfiguration.

Representative DDS waveforms are shown in Figure 6. The waveform capture includes a dynamic FTW update demonstrating runtime frequency reconfiguration.

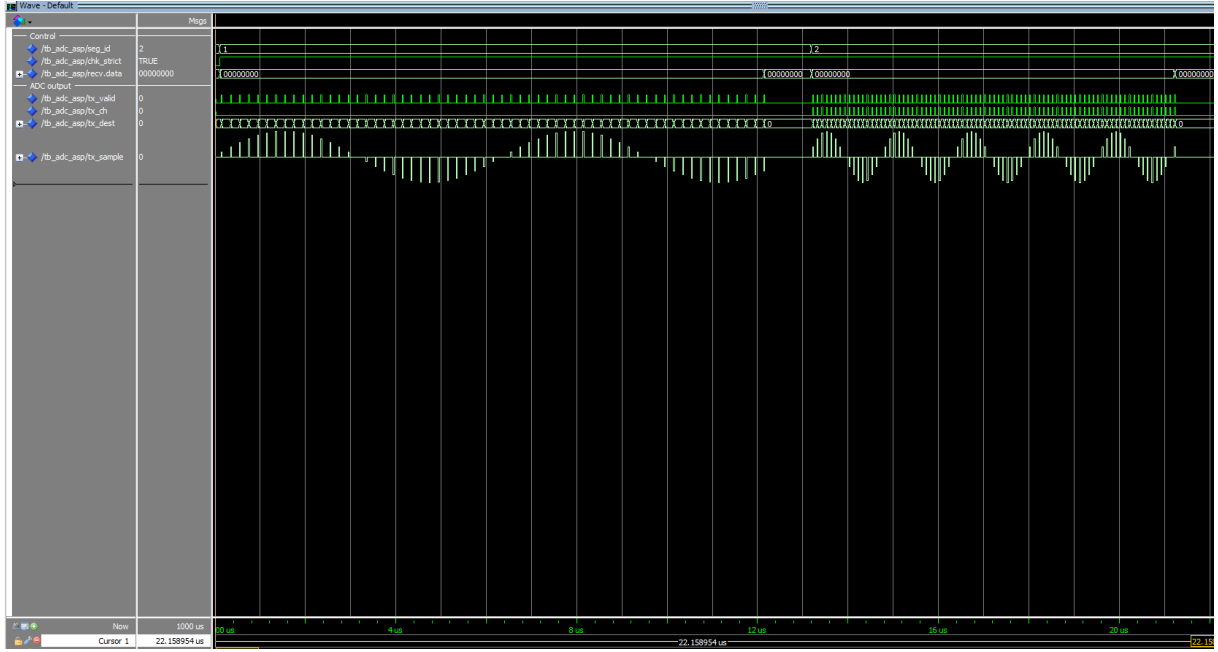


Figure 6: `tb_adc_asp` simulation: generated DDS sine output, including a live FTW update demonstrating runtime frequency reconfiguration.

Both ASPs close timing comfortably against the 50 MHz board clock, with roughly $2.3\text{--}2.5\times$ margin (Table 3), confirming that neither datapath is timing-critical on the target device.

Several architectural trade-offs were considered during the design process.

The AVG-ASP uses a running-sum architecture rather than recomputing the full moving-average sum for every sample. This reduces the datapath cost to one addition and one subtraction per sample regardless of the selected window length.

The averaging window was implemented using shift-register storage rather than RAM-based circular buffers. For the relatively small maximum window size of 16 samples, this approach simplifies timing and control logic while using only modest register resources.

For the ADC-ASP, the selected LUT depth represents a trade-off between waveform quality and memory usage. Increasing the LUT size improves waveform precision but increases ROM resource consumption.

The DDS frequency resolution is determined by the phase accumulator width. Increasing the accumulator width would improve frequency precision at the cost of additional logic resources.

9 Hardware versus Software Implementation

A C reference model was implemented for the AVG-ASP to describe the equivalent software behaviour. The model uses the same running-sum algorithm as the hardware design, including WARMUP behaviour, bypass mode, and power-of-two division using arithmetic right shifts. This provides a direct software baseline for comparing the RTL implementation against an equivalent sequential implementation.

The moving-average filter can be implemented straightforwardly in software using a running-sum algorithm. However, a software implementation on a general-purpose processor executes sequentially and consumes processor cycles for every received sample.

In contrast, the AVG-ASP implements the same algorithm directly in hardware using dedicated registers and arithmetic datapaths. Once configured, the ASP processes streaming input data autonomously without software intervention.

The hardware implementation provides deterministic throughput of one result per input sample after the WARMUP phase. The running-sum datapath also avoids repeated summation of the entire averaging window, reducing the arithmetic cost to one addition and one subtraction per sample independent of the selected window length.

```

1 typedef struct {
2     int16_t window[MAX_L];
3     int32_t sum;
4     int count;
5     int index;
6     int L;
7     int shift;
8     bool bypass;
9 } AvgRef;
10
11 void avg_init(AvgRef *avg, int mode) {
12     avg->sum = 0;
13     avg->count = 0;
14     avg->index = 0;
15     avg->bypass = false;
16
17     for (int i = 0; i < MAX_L; i++) {
18         avg->window[i] = 0;
19     }
20
21     switch (mode) {
22         case 0: avg->bypass = true; avg->L = 1; avg->shift = 0; break;
23         case 1: avg->L = 4; avg->shift = 2; break;
24         case 2: avg->L = 8; avg->shift = 3; break;
25         case 3: avg->L = 16; avg->shift = 4; break;
26         default: avg->bypass = true; avg->L = 1; avg->shift = 0; break;
27     }
28 }
29
30 // Returns true when output is valid; false during WARMUP.
31 bool avg_process_sample(AvgRef *avg, int16_t x, int16_t *y) {
32     if (avg->bypass) {
33         *y = x;
34         return true;
35     }
36
37     int16_t old = avg->window[avg->index];
38
39     avg->window[avg->index] = x;
40     avg->index = (avg->index + 1) % avg->L;
41
42     if (avg->count < avg->L) { // WARMUP
43         avg->sum += x;
44         avg->count++;
45
46         if (avg->count < avg->L) {
47             return false;
48         }
49
50         *y = (int16_t)(avg->sum >> avg->shift);
51         return true;
52     }
53
54     avg->sum += x - old; // running sum
55     *y = (int16_t)(avg->sum >> avg->shift);
56     return true;
57 }

```

Listing 1: Core of the AVG-ASP C reference model (ip/software/avg_ref.c).



Figure 7: `testbed_pipeline` simulation: end-to-end ADC→AVG flow over the TDMA-MIN NoC, showing pre- and post-filter samples and moving-average attenuation of the sine.

Compared with a software implementation, the hardware ASP reduces processor workload and enables continuous real-time filtering while allowing the Nios II processor to perform higher-level application tasks.

The ADC-ASP waveform generator could also be implemented in software by periodically calculating or retrieving waveform samples and transmitting them through the NoC. However, implementing the DDS generator in hardware allows waveform generation to proceed continuously and independently of processor execution.

The hardware DDS implementation also enables runtime frequency reconfiguration through packet-based FTW updates without interrupting waveform generation or requiring software-managed timing loops.

10 Parallelisation Opportunities

The implemented ASPs already exploit a moderate degree of hardware parallelism. The ADC-ASP contains two independent DDS channels operating concurrently, while the AVG-ASP maintains independent filtering state for each audio channel.

In the current system, overall throughput is limited primarily by the TDMA-MIN communication bandwidth rather than computational complexity within the ASP datapaths. The NoC permits only one packet transmission per slot, meaning communication becomes the dominant bottleneck before arithmetic throughput limits are reached.

The AVG-ASP running-sum implementation already achieves constant-cost filtering independent of window length. For substantially larger filters or more complex FIR implementations, additional pipelining or parallel adder structures could be introduced to improve throughput.

Future scalability improvements could also include FIFO-buffered packet transmission or wider NoC datapaths to support higher concurrent sample rates.

11 Conclusion

This Individual Project delivered two composable, NoC-connected hardware accelerators for the power-system frequency-measurement pipeline: an ADC-ASP that emulates a sampled analogue front-end using Direct Digital Synthesis, and an AVG-ASP that implements a programmable moving-average filter using a resource-efficient

running-sum datapath. Both modules integrate directly into the reused Lab-2 TDMA-MIN NoC as drop-in nodes, requiring no reset or node-identity ports, and communicate through a shared 32-bit packet protocol.

The DDS architecture allows the emulated grid frequency to be retuned at runtime through a single configuration packet, and the moving-average filter supports window lengths of 4, 8 and 16 using only power-of-two arithmetic shifts, with no hardware divider or multiplier. The running-sum architecture keeps the filtering cost at one addition and one subtraction per sample regardless of window length, and a dedicated WARMUP phase guarantees that every emitted output is a mathematically complete average.

Functional correctness was established through self-checking unit testbenches that compare the implementation against independent reference calculations, and through system-level testbeds that exercise the modules over the real TDMA-MIN NoC, including end-to-end pipeline behaviour and packet-routing correctness. Synthesis confirms both ASPs are small and fast, using no DSP blocks and closing timing with roughly $2.3\text{--}2.5\times$ margin against the 50 MHz board clock.

The principal remaining work is the completion of the open team-level design decisions described in Section 5 (sample-rate table, Mode encoding, and the 8-port node-ID map), and integration of the two ASPs with the remaining pipeline stages and the ReCOP subsystem in GP-2. The current results demonstrate that hardware-accelerated DSP kernels can sustain deterministic, real-time throughput far in excess of the system's communication bandwidth, validating the ASP-based approach for the critical part of the HMPSoC.

12 Acknowledgements

The design includes files supplied by the University of Auckland and Zoran Salcic in Lab-2.